Spotify: Installation & User Manual

A Spotify-like music app in pure Java sockets, with a website and a terminal client on a 3-node backend

1. Requirements

- **JDK 11 or newer.** Check with `javac -version` (and `java -version`). No other software is needed: no database, Maven or npm.
- A modern web browser (Chrome, Firefox or Edge) for the website.
- Free network ports on your own computer: **8080** (website) and **5001, 5002, 5003** (the three backend nodes). No internet connection is required.

2. Installation

1. Unzip the project. Open a terminal **inside the project folder**, the one that contains `run.bat`, `run.sh`, `client/`, `server/` and `public/`.
2. Build the project (first time only, and again after any code change):

```
Windows:      run.bat build
Linux / Mac:  ./run.sh build        (if needed first: chmod +x run.sh)
```

3. You should see **Build OK**. The compiled classes are placed in `server/out`.

3. Starting and Stopping

Goal	Windows	Linux / Mac	What happens
Start everything	`run web`	`./run.sh web`	Starts the 3-node backend, then the website. Open **http://localhost:8080**. The first run also generates 18 WAV songs (about 5 s).
Terminal client (optional)	`run terminal`	`./run.sh backend` `./run.sh terminal`	Command-line Spotify. Needs the backend running, so start `run web` first. Can run together with the website.
Backend only (optional)	`run backend`	`./run.sh backend`	Starts only the 3 nodes, for use with the terminal client alone.

- **Windows:** the backend opens in a second window called "SimpleStream Backend". **Linux / Mac:** it runs in the background. **To stop:** close the windows (Windows) or press Ctrl+C (Linux / Mac).
- Without the scripts: `java -cp server/out simplestream.MusicServer` (backend), then `java -cp server/out simplestream.SpotifyWeb` (website).

4. Using the Website

1. **Sign Up:** choose the Sign Up tab. Enter an email, a username (3–15 letters, digits or _) and a password (at least 3 characters), then press **Create account**.
2. **Log In:** you are taken to the Log In tab with the "Account created" message. Log in with your **email and password** (not your username). A page refresh keeps you logged in.
3. Use the **sidebar** to move around:

Screen	What you can do
Home	Recently played songs, most-played songs and picks for each language.
Search	Type a title, artist or language (Hindi / English / Telugu); results appear as you type.
Library	Songs you liked. Click the heart on any song card to add or remove it.
Playlists	Create a playlist (name, then Create). Click a playlist to open it; use "+ Playlist" on any song to add it, the cross (X) to remove it, and the delete button to remove the playlist.
All Songs	The full catalogue of 18 songs, grouped as Hindi, English and Telugu.

- **Playing:** click a song card (or its round play button). The **player bar** at the bottom has Play/Pause, Previous, Next and a seek bar, and stays visible on every screen.
- **Sidebar footer:** shows the 3 backend nodes (green = up, red = down, with the node that served the last request), your username, **Log out**, and the **Crash last node** demo button.

5. Using the Terminal Client

Start it with `run terminal`. The first screen asks you to sign up or log in with a **username and password** (no email). Terminal sound plays **WAV only**. After each request it prints which node served it and the logical clock.

Group	Commands
Account	`signup <user> <pass>` | `login <user> <pass>` | `logout` | `whoami`
Browse	`home` | `songs` | `search <text>` | `library` | `like <songId>`
Playlists	`playlists` | `open <playlist>` | `create <name>` | `delete <name>` | `add <songId> <playlist>` | `remove <songId> <playlist>`

Group	Commands
Playback	`play <songId>` | `pause` | `resume` | `next` | `prev` | `now`
Cluster / other	`nodes` (show cluster) | `crash <port>` (simulate a node failure) | `help` | `quit`

Example session:

```
signup alice 1234 login
alice 1234
songs                  (list songs and their ids)
play 7                 (play song 7) like 7
create road add 7 road
nodes                  (see which nodes are up) quit
```

6. Try the Distributed Features

Demo	Steps	What you should see
Load balancing	Click through several website pages, or run a few terminal commands.	Sidebar “last served” marker moves 5001 → 5002 → 5003; the terminal prints “served by node …”.
Fault tolerance	Click **Crash last node** (website) or type `crash 5001` (terminal), then keep using the app.	The crashed node turns red, yet every action still works through the other nodes.
Replication	Like a song or create a playlist, then keep browsing so different nodes answer.	The same Library and playlists appear whichever node replies.
Persistence	Stop the backend, start `run web` again, and log in.	Accounts, likes, playlists and history are still there. A crashed node returns only after a full backend restart.
Lamport clock	In the terminal, run `songs`, then `like 7` and `like 8`, and read “logical clock N” after each reply.	Every write raises the serving node's clock; reads leave it unchanged. Replicas jump to max(own, received) + 1.
Replication in the log	Watch the backend window while you like a song. (Windows: “SimpleStream Backend”; Linux / Mac: the terminal running `run.sh web`.)	One line from the receiving node, e.g. `[Node 5002] LIKE clock=5`, then `(replica)` lines from the other two nodes.

Same account, two clients	Sign up on the website. In the terminal run `login <username> <password>`, then `library`. Like a song in the terminal and refresh the website Library.	Both clients show the same data, although different nodes answer each request.
Several failures	Crash nodes one at a time (button or `crash <port>`) and keep using the app. Run `nodes` to see the status.	Works with one node left. The backend log shows "replica … is down - skipped"; the website window shows "[Balancer] node … not reachable". Crash the last node and the app reports "All backend nodes are down"; restart `run web`.
Stateless Next / Prev	Play a song from a playlist, crash the node that last served you, then press Next or Previous.	Still works: each request carries the song and playlist, so any node can answer.
Concurrent users	Log in as two different accounts in two browsers, plus the terminal, and play, like and edit playlists at the same time.	All work at once (one thread per connection); each user sees only their own library and playlists.

7. Using Your Own Music

Song files are named by the song's **id** (its position in `Catalog.java`, starting at 1). Put `7.mp3` in `server/data/audio/`, **delete the matching** `7.wav` (the website plays the WAV first), edit that song's title and artist in `Catalog.java`, run the build and restart. To add new songs, append lines at the **end** of the `SONGS` array in `Catalog.java` (ids 19, 20, …) and add `19.mp3`, `20.mp3`. Never insert songs in the middle: it changes later ids and mixes up saved likes. The website plays mp3 and wav; the terminal client plays only WAV.

8. Troubleshooting

Problem	**Fix**
"Address already in use"	An old backend is still running. Close its window or terminal and start again.
"All backend nodes are down. Start MusicServer first."	The backend is not running. Start it with `run web` (or `run backend` for the terminal client).
`javac` is not recognised	Install JDK 11+ (not just a JRE) and make sure it is on your PATH, then open a new terminal.
Cannot log in on the website with my username	The website logs in with **email + password**. Accounts created in the terminal have no email; log in to those from the terminal.
Start again from scratch	Stop the app and delete `server/data/users.db` and `server/data/userdata.db`. They are re-created automatically.

- **23MCCE22 (K AMULYA)**